

操作系统学习总纲



最后更新：2026-05-30



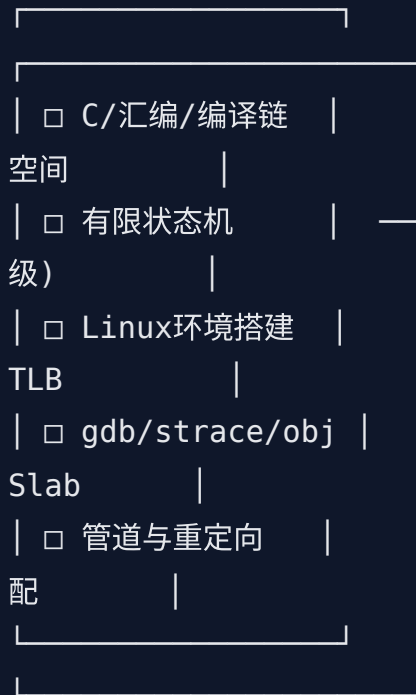
基于 38 天系统学习笔记提炼，持续迭代中

一、学习路线图（全局视图）

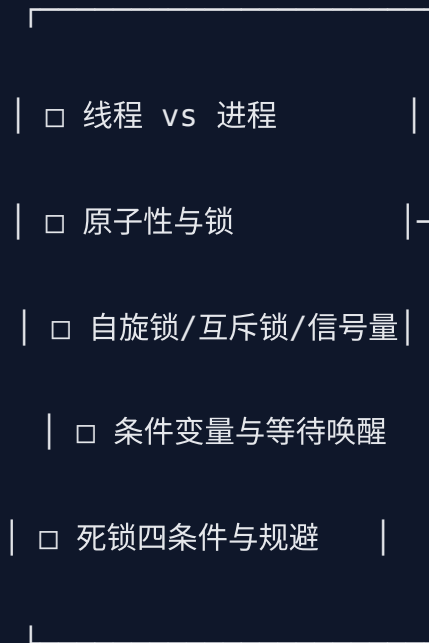
操作系统学习全景路线

阶段0：基础启蒙

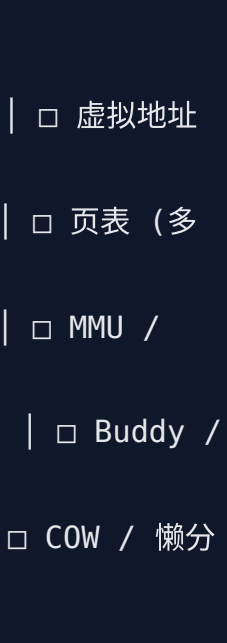
存管理



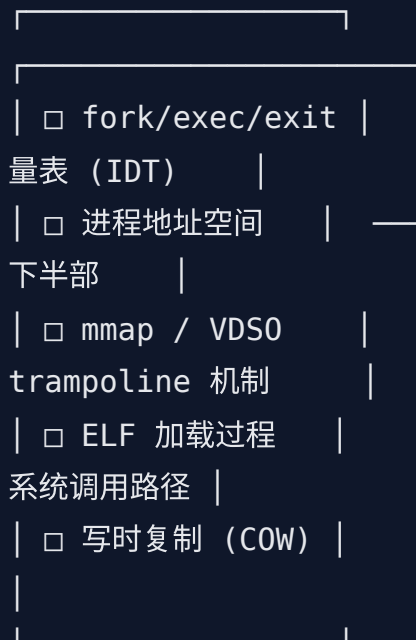
阶段1：并发与同步



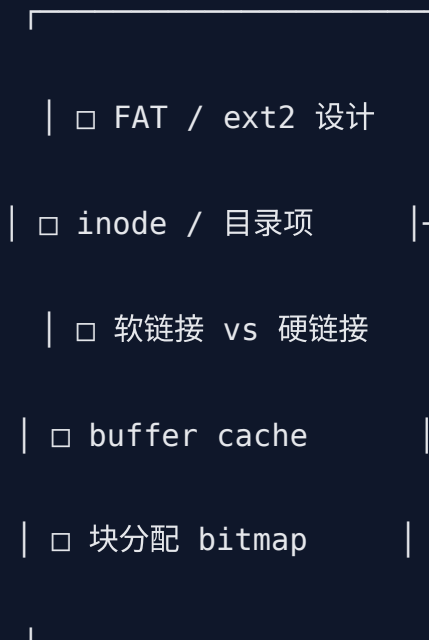
阶段2：内



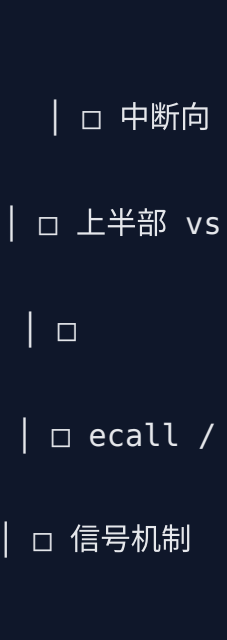
阶段3：进程与系统调用与Trap



阶段4：文件系统



阶段5：中断





路线图说明

以上是**主线路径**，适合按阶段递进学习。每个阶段之间有依赖关系（箭头标注），但阶段内部的知识点可以交叉学习。如果你已经有一定基础，可以直接跳到感兴趣的阶段。

扩展提示： 后续可以增加的方向包括——设备驱动模型（I2C/Platform 总线）、网络协议栈、虚拟化技术、实时操作系统（RTOS）等。添加时只需在对应阶段下补充即可。

二、各阶段重点（关键词 + 注意点）

阶段0：基础启蒙

关键词	必须掌握	注意事项
有限状态机 (FSM)	程序本质 = 状态机；C→汇编→二进制由编译器完成	⚠ 编译优化可能合并/删除状态；语义上有效的状态不能被错误优化
Linux 环境	WSL2、gdb（单步/断点/watchpoint）、strace（系统调用跟踪）	⚠ static 链接会使 objdump 分析变困难，调试时优先动态链接

编译工具链	objdump -d -S (需 -g)、 readelf -h/-l、size、nm	⚠️ -ffreestanding 下仍可用 stddef.h / stdint.h / stdbool.h 等
Shell 基础	> 输出重定向, | 管道 (本质是 IPC)	⚠️ stdout 到终端是 line buffer, 到 pipe/file 是 full buffer
大小端	小端: 低字节在低地址 (主流); 大端: 低字节在高地址	⚠️ 网络字节序通常是大端
复杂声明	int *a[5] 指针数组 vs int (*a)[5] 数组指针; long (*fp)(int) 函数指针	⚠️ 不要记"最后和什么结合", 而是按 C 声明语法解析

阶段1: 并发与同步

关键词	必须掌握	注意事项
线程 vs 进程	线程共享地址空间 (全局变量共享), 各有独立栈; 进程有独立地址空间 (需 MMU)	⚠️ volatile 只防编译器优化, 不能保证线程安全
原子性	多线程破坏原子操作 → 需要锁或原子指令 (lock 前缀、xchg)	⚠️ 原子指令的"读-改-写"整体不会被其他核打断
内存一致性	多核乱序 + Cache → 需要内存屏障 (mfence)、__sync_synchronize()	⚠️ 即使单条指令原子, 也可能因 cache coherence 导致可见性问题
自旋锁 (spinlock)	忙等, 适合临界区短、竞争少的场景; 持有期间关中断	⚠️ 自旋锁内不能休眠! 单核上必须关中断防止死锁
互斥锁 (mutex)	睡眠锁, 竞争失败进入休眠; Futex = fast path (用户态原子操作) + slow path (系统调用)	⚠️ 中断上下文绝对不能睡眠 (无进程上下文)
条件变量	cond_signal 唤醒一个, cond_broadcast 唤醒全部	⚠️ 等待后必须用 while 重新检查条件, 防止虚假唤醒
信号量 (semaphore)	P/V 操作 (wait/post), 二值信号量 ≈ 互斥锁	⚠️ mutex 有所有权语义 (谁拿谁放), 信号量无此限制, 优先用 mutex

死锁	四必要条件：互斥、请求与保持、不剥夺、循环等待	⚠ Lock ordering（固定顺序申请锁）是最实用的死锁预防方法
读写锁	多读者并发，写者互斥；需写者优先防饥饿	⚠ 临界变量必须在锁内修改，混用锁和原子操作极易出 bug
Peterson 算法	store/load 实现两线程互斥："举旗 + 贴纸条"，先到先得	⚠ 多核上仍需内存屏障
Per-CPU 优化	每个 CPU 独立 freelist，减少锁竞争	⚠ 偷取时 先释放自己锁再拿别人锁 ，否则死锁；注意断链操作

阶段2：内存管理

关键词	必须掌握	注意事项
虚拟地址空间	4GB（32位），各段： text/data/bss/stack/mmap/heap	⚠ pmap 读 /proc/pid/maps，可与 readelf -l 互相印证
页表 (Page Table)	Sv39: 9+9+9+12 = 39 位 VA；PTE 含 PPN + flags (V/R/W/X/U)；多级减少内存开销	⚠ 内核操作的是虚拟地址（x86 恒等映射只是特例），不是物理地址
MMU / TLB	MMU 自动遍历页表翻译 VA→PA；TLB 缓存最近 PTE	⚠ 切换进程 = 切换页表基址寄存器（如 satp / CR3），需刷新 TLB
Buddy Allocator	管理 2 的幂大小页块（4K→16M），分裂/合并用 XOR 算 buddy	⚠ pool_start 必须对齐到最大块大小，否则对齐要求无法满足
Slab Allocator	一页切多块小对象（16B→2048B），free_list 用 intrusive list	⚠ 空闲对象的前几字节存 next 指针，分配后覆盖——不需要额外内存
COW (写时复制)	fork 时共享页表项、清 PTE_W；写时缺页异常触发真正复制	⚠ 需维护引用计数，归零才真释放；父进程 PTE 也要清写权限！

mmap	建立虚拟映射，懒分配——缺页时才分配物理页	⚠️ <code>mmap</code> 不立即分配内存； <code>malloc</code> 底层可能用 <code>mmap</code> + <code>sbrk</code>
VDSO / VVAR	只读/高频系统调用不陷入内核（如 <code>gettimeofday</code> ），映射到每个进程	⚠️ 类似 xv6 中 trampoline 页在所有进程中映射
SRAM vs DRAM	SRAM：触发器，快，贵，做 Cache；DRAM：电容，需刷新，做主存	⚠️ SSD 内有自己的 FTL（逻辑→物理映射），类似虚拟内存的思想

阶段3：进程与系统调用

关键词	必须掌握	注意事项
fork()	复制进程，COW 优化；子进程返回0，父进程返回子PID	⚠️ 多重 fork 画状态机分析，关注 PC 位置和 PID 返回值
execve()	替换进程映像， <code>_start</code> → <code>__libc_start_main</code> → <code>main</code>	⚠️ She-bang (<code>#!</code>) → <code>execve</code> 的变参数替换
exit()	<code>exit(0)</code> 调用 <code>atexit</code> 注册函数 + 清理； <code>_exit(0)</code> 直接退出	⚠️ 多线程 <code>fork()</code> 后只有调用线程被复制，其他线程消失
文件描述符	fd 是打开文件表索引；0/1/2 是 <code>stdin/stdout/stderr</code>	⚠️ fork 后父子共享 open file description (offset 共享)
进程组	终端信号发到前台进程组；fork 后继承 PGID	⚠️ <code>Ctrl+Z</code> 挂起 (<code>SIGTSTP</code>)， <code>Ctrl+C</code> 中断 (<code>SIGINT</code>)，fg/bg 切换
ELF 文件	ELF Header → Program Headers → <code>.text/.data/.bss</code> → 符号表	⚠️ 静态链接体积大但简单；动态链接 <code>.so</code> 共享但依赖 <code>ld-linux.so</code>
栈帧结构	fp-8 存返回地址 ra，fp-16 存上级帧指针	⚠️ backtrace 实现：沿 fp 链回溯，边界条件不得超过页范围
vfork	父子共享地址空间，子进程先运行父进程阻塞	⚠️ vfork 后必须立即 <code>execve</code> 或 <code>_exit</code> ，否则危险

阶段4：文件系统

关键词	必须掌握	注意事项
-----	------	------

文件系统本质	磁盘上的数据结构，按块访问，block set → 变长数组 → 文件/目录	⚠️ 核心假设是"按块访问"，随机访问靠索引结构
FAT 文件系统	目录项存文件名+起始簇号；FAT 表存链表（next cluster）；简单但易碎片	⚠️ FAT 表损坏 = 数据丢失，因此通常有多份备份
ext2 / UNIX FS	inode 集中存元数据+数据块指针；目录项是"文件名→inode号"映射	⚠️ inode 本身不存文件名，这允许多个硬链接指向同一 inode
inode 三形态	dinode（磁盘原始结构）→ inode（内存版，加锁/引用计数）→ stat（用户态精简投影）	⚠️ <code>ls -li</code> 看 inode 号； <code>ln</code> 创建硬链接（共享 inode）， <code>ln -s</code> 创建软链接
数据块索引	direct blocks → indirect block → 多级类似页表	⚠️ 大文件间接索引，小文件直接块，需要权衡
xv6 文件系统布局	[boot / sb / log / inodes / bitmap / data]	⚠️ <code>bio.c</code> 实现 buffer cache → <code>bget</code> 接口
内部 vs 外部碎片	内部碎片 = 分配单元大于实际使用；外部碎片 = 空闲空间不连续	⚠️ 小文件多时 FAT 碎片化严重，ext2 通过 inode 集中存储改善
挂载 (mount)	将设备文件系统挂到 VFS 目录树；任意目录可做挂载点	⚠️ 磁盘镜像文件可通过 loop device 再 mount（文件中的文件系统）

阶段5：中断与 Trap

关键词	必须掌握	注意事项
Trap 四类	系统调用、设备中断、缺页异常、时钟中断	⚠️ 发生 trap 时，正在执行的指令应当可恢复并继续
ecall / 系统调用路径	用户 ecall → trampoline.S (uservec) → 切换内核页表 → usertrap() → syscall() → 返回	⚠️ 系统调用号在 a7 寄存器，返回值写回 trapframe->a0
trampoline 页	同时映射在用户页表和内核页表最高处，用于 trap 切换	⚠️ RISC-V trap 不自动切换页表，所以 trampoline 必须在两个页表都有映射

中断上下文	上半部（硬中断）：关中断、快处理；下半部（softirq/tasklet/workqueue）：延迟处理	⚠️ 工作队列 = 内核线程 → 可睡眠；softirq/tasklet 不能睡眠
IDT (中断描述符表)	中断号 → 处理函数地址；CPU 自动保存 CS/RIP/RFLAGS/SS/RSP	⚠️ x86 进入中断时硬件自动压栈这些寄存器
信号机制	SIGINT/SIGKILL/SIGSEGV 等；进程可注册 handler；内核发送信号	⚠️ Ctrl+C → 终端向前台进程组发 SIGINT
自旋锁 + 中断	拿自旋锁必须关本地中断，防止死锁	⚠️ 持锁时被中断 → 中断处理程序尝试拿同一把锁 → 死锁
RISC-V 特权级	M-mode（机器态）→ S-mode（内核态）→ U-mode（用户态）；各有独立的 CSR	⚠️ ARMv7 有9种模式，IRQ 有独立寄存器副本，不需立即保存用户栈

三、学习遇到的坑（含解决方案）

🔴 多线程编程坑

编号	坑描述	根因	正确做法
坑 1	释放锁后才读取共享数据	释放锁后槽位已被其他线程覆写	在锁内把数据复制到局部变量，再释放锁
坑 2	多个 worker 重复处理同一任务	看到槽位有数据就直接处理，未标记已取走	在锁内标记该槽位为"空"，再释放锁
坑 3	程序结束时 worker 死等	输入线程只 signal 一个 worker，其他 worker 永远睡眠	输入结束时用 <code>pthread_cond_broadcast</code> 唤醒所有等待线程
坑 4	<code>cond_wait</code> 后用 <code>if</code> 而非 <code>while</code>	虚假唤醒或条件被其他线程抢先改变	始终用 <code>while</code> 重新检查条件
坑 5	线程入口函数相同则误以为共享栈	对线程模型理解不足	每个线程有独立栈/PC/寄存器，仅共享全局变量和堆

● 内存管理坑

编号	坑描述	根因	正确做法
坑 6	metadata 写成超大静态数组	<code>pages[1048576]</code> 静态分配 32MB，超 1MB 限制	在 <code>pmm_init()</code> 中从 heap 开头切 metadata
坑 7	<code>pool_start</code> 未对齐到 16MiB	大块分配的对齐要求无法满足	初始化时 <code>align_up(hs + meta_bytes, MAX_ALLOC)</code>
坑 8	COW 只改了子进程 PTE	忘记父进程对应页仍可写	fork 时父子进程 PTE 都要清 PTE_W + 设 COW 标记
坑 9	COW 标志位覆盖其他权限位	<code>flags &= 1<<8</code> 清空了除 COW 外的全部 flag	使用 <code>`flags</code>
坑 10	double free 无法检测	没有校验传入地址的有效性	可加 assert: <code>`assert(pages[id].flag == PG_SLAB</code>
坑 11	slab 页永远不归还	<code>inuse==0</code> 时不释放整个页	空 slab 页还给 buddy: <code>buddy_free_pages(page_addr(id))</code>

● 并发/锁机制坑

编号	坑描述	根因	正确做法
坑 12	CPU1 偷 CPU2 同时 CPU2 偷 CPU1 → 死锁	持自己锁时尝试拿对方锁，形成循环等待	先释放自己锁，再去偷别人的
坑 13	偷链表后半段未正确断链	修改了整个 freelist，但原链表仍有引用	只修改中间节点的 next 指针，不要整链表置空
坑 14	锁保护变量和原子操作混用	部分状态在锁外修改，产生竞态	所有临界变量的读写 必须都在持锁状态下
坑 15	自旋锁内调用可能休眠的函数	spinlock 持有期间休眠会导致死锁	自旋锁内绝不调用可能 block 的函数

● 系统编程坑

编号	坑描述	根因	正确做法
坑 16	内核态直接使用用户空间地址	用户虚拟地址在内核页表中无效	使用 <code>copyin / copyinstr</code> 或 <code>argstr / argint</code> 安全拷贝
坑 17	alarm 返回后不知原执行位置	直接修改 <code>trapframe->epc</code> 跳转到 handler，丢失了原始上下文	先保存完整 <code>trapframe</code> 副本， <code>alarm_return</code> 时恢复
坑 18	<code>alarm_return</code> 覆盖系统调用返回值	恢复旧 <code>trapframe</code> 时冲掉了当前 <code>a0</code>	先保存 <code>a0</code> → 恢复旧 <code>trapframe</code> → 写回 <code>a0</code>
坑 19	<code>find</code> 递归时直接修改路径指针	破坏了原始路径，递归上层使用错误	创建 <code>buf[]</code> 副本，用独立指针 <code>p</code> 定位尾部
坑 20	<code>strcpy</code> 到指针数组的元素前未分配内存	野指针	<code>char *p[num]</code> 中每个 <code>p[i]</code> 必须先指向有效内存

● 工具链/调试坑

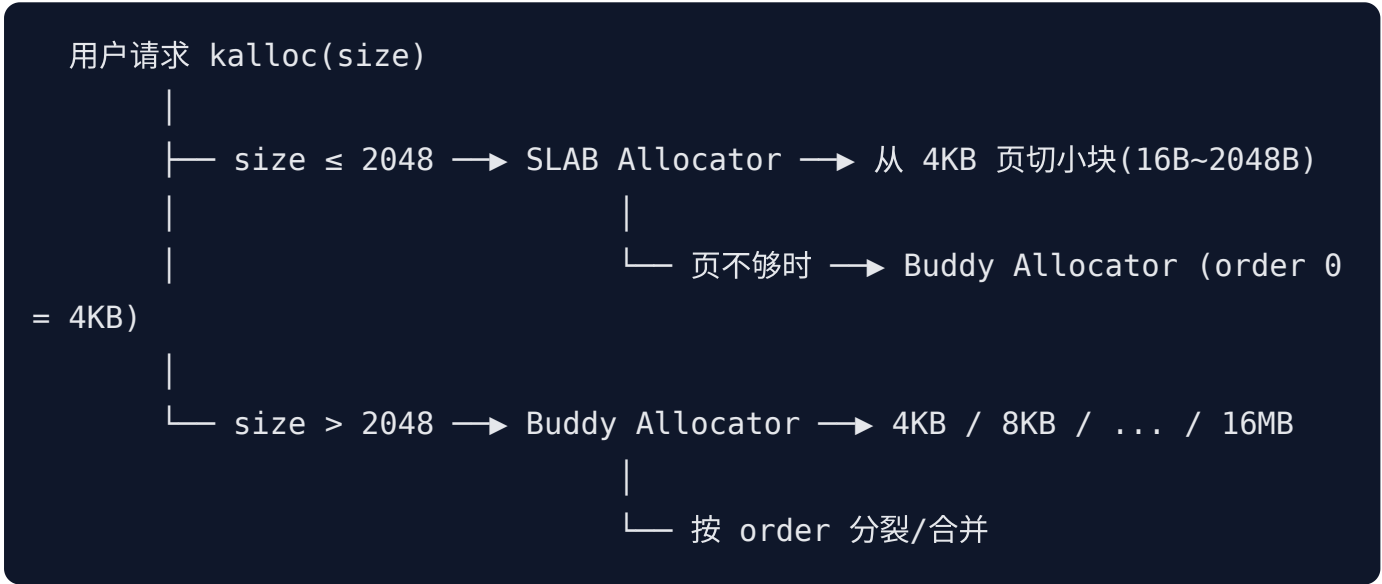
编号	坑描述	根因	正确做法
坑 21	<code>strace</code> 输出和程序输出混在一起	两者都输出到同一终端	用 <code>strace -f -o /tmp/log ./prog</code> 分开输出
坑 22	WSL2 缺少内核源码/headers 无法编译 <code>.ko</code>	WSL2 默认无完整内核构建环境	安装对应内核 headers 或在完整 Linux 环境中编译
坑 23	<code>\n</code> 在串口输出格式不对	某些终端需要 CR+LF	用 <code>\r\n</code> 代替 <code>\n</code>

四、关键概念速查表

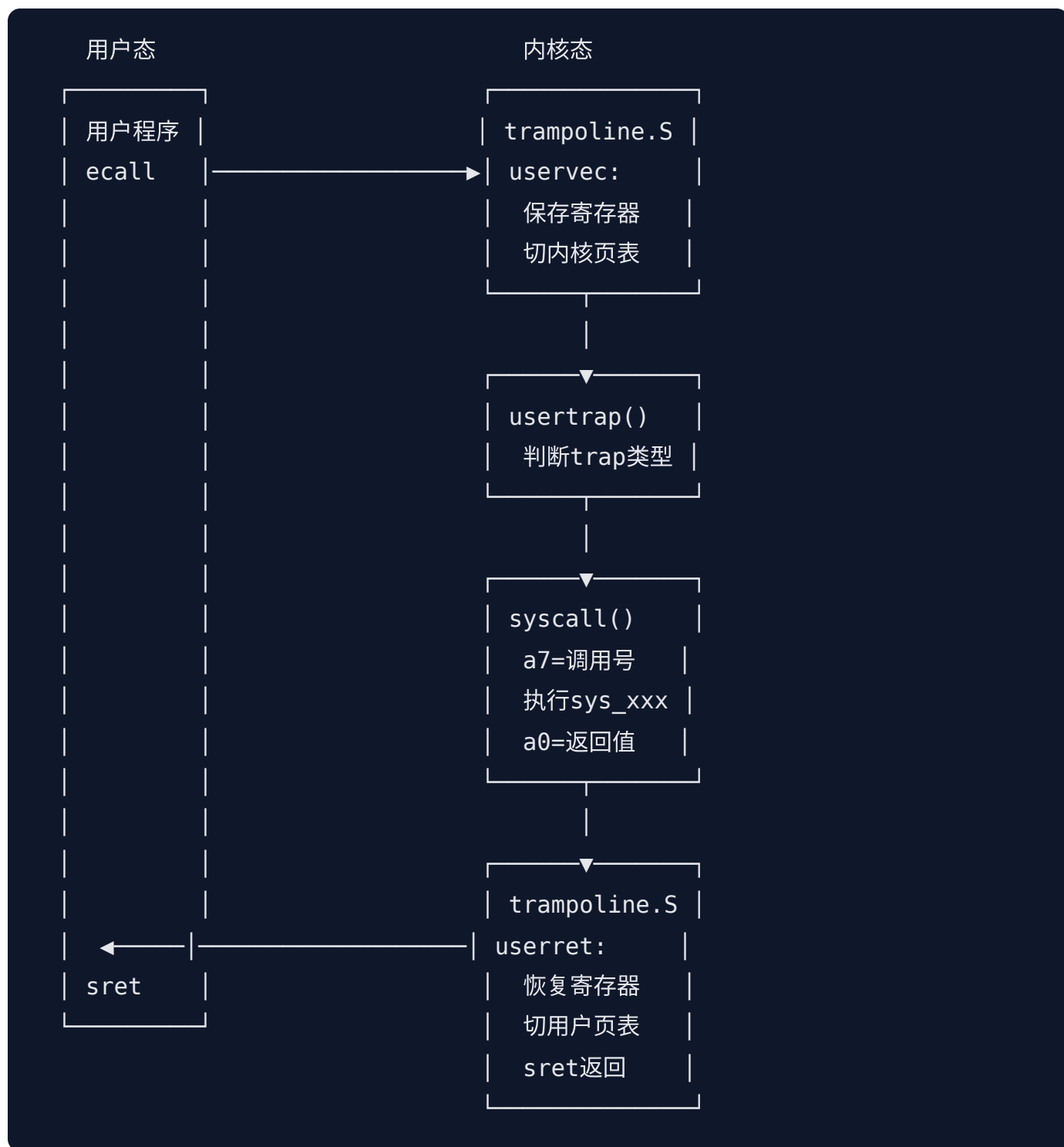
4.1 锁类型对比

	自旋锁(spinlock)	互斥锁(mutex)
等待方式：	忙等(循环)	休眠(不占CPU)
系统调用：	无	有(Futex slow)
适合场景：	临界区短+竞争少	临界区长+竞争多
中断上下文：	可用(关中断)	❌ 绝对不能用
持有期间：	不能休眠	可以休眠
加锁开销：	低(无syscall)	较高(需syscall)

4.2 内存分配层次



4.3 系统调用路径全景



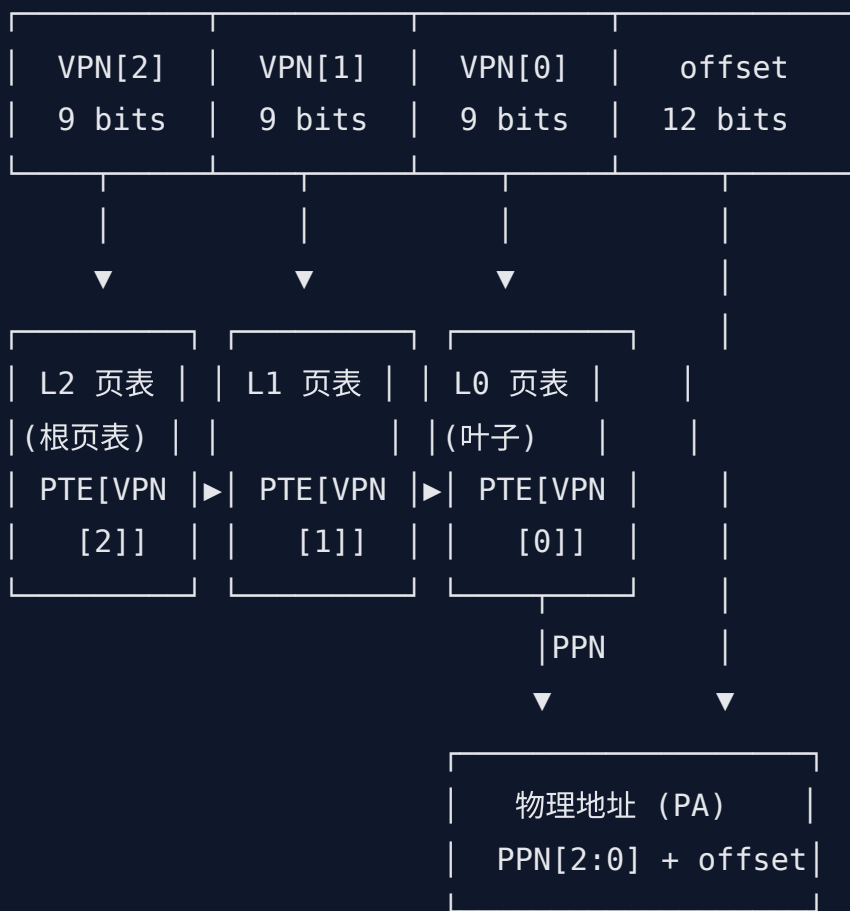
4.4 ELF 文件加载流程

磁盘 ELF 文件 → execve() → 内核 loader



4.5 多级页表地址翻译 (Sv39)

虚拟地址 (39 bits)



五、总复习策略

5.1 三层复习法



5.2 可视化复习工具建议

工具	用途	方式
手绘流程图	系统调用路径、中断处理流程、fork/exec 流程	默画，对照笔记矫正
对比表格	锁类型对比、文件系统对比、分配器对比	遮住一半默写另一半
心智模型图	进程地址空间布局、文件系统布局	画到能脱稿讲解
代码走读	xv6 关键路径 (trap、syscall、fork、COW)	拿张纸手写关键函数调用链

5.3 复习检查清单

- ❑ 能默画出系统调用完整路径 (ecall → trampoline → usertrap → syscall → 返回)
- ❑ 能解释 fork + exec 过程中虚拟地址空间如何变化
- ❑ 能对比自旋锁、互斥锁、读写锁、信号量的异同
- ❑ 能手写 buddy allocator 的分裂与合并逻辑 (至少伪代码级别)
- ❑ 能画出多级页表地址翻译过程
- ❑ 能解释 COW 从 fork 到缺页异常的完整流程
- ❑ 能对比 FAT 和 ext2 的设计差异及各自优劣
- ❑ 能写出生产者-消费者模型的正确线程同步代码
- ❑ 能列举至少 5 个自己踩过的坑及根因
- ❑ 能解释为什么中断上下文不能睡眠

5.4 常见面试/考试切入点

问题	涉及知识点
"线程和进程的区别"	阶段1 + 阶段3
"虚拟内存是怎么工作的"	阶段2 (VA→PA, 页表, TLB, 缺页)
"malloc(16) 底层发生了什么"	阶段2 (slab → buddy → mmap → 缺页)
"自旋锁和互斥锁什么时候用哪个"	阶段1 + 阶段5 (中断上下文)
"fork() 之后父子进程共享什么"	阶段3 (COW, 文件描述符, 地址空间)
"死锁怎么产生的, 怎么避免"	阶段1 (四条件 + lock ordering)
"系统调用是怎么陷入内核的"	阶段5 (ecall → trap → trampoline)
"文件系统怎么找到文件数据"	阶段4 (目录项 → inode → 数据块)

六、扩展建议

💡 以下内容可在后续学习中逐步添加, 不会破坏现有结构。

待补充方向

- ❑ **设备驱动模型**: I2C 驱动框架 (platform bus + I2C bus)、字符设备注册流程

- ☐ 网络协议栈：socket → TCP/IP → 驱动层的数据路径
- ☐ 虚拟化技术：VM、容器、hypervisor 基本原理
- ☐ 实时操作系统 (RTOS)：优先级反转、死锁检测、确定性调度
- ☐ Linux 内核机制深入：RCU、完全公平调度器 (CFS)、cgroup 资源隔离
- ☐ 性能分析工具链：perf、ftrace、eBPF 基础
- ☐ 安全机制：ASLR、Stack Canary、KASLR、seccomp

添加新内容的格式建议

当需要添加新的知识点时，建议遵循以下格式以确保一致性：

```
### 阶段X：[主题名称]

| 关键词 | 必须掌握 | 注意事项 |
|-----|-----|-----|
| **概念1** | 核心描述 | ⚠️ 常见误区 |
| **概念2** | 核心描述 | ⚠️ 常见误区 |
```

七、参考资料索引

资源	用途	备注
xv6-riscv 源码	操作系统实现参考	kernel/ 内核代码， user/ 用户程序
xv6 Book	配套说明文档	每章对应一个子系统
man 手册 (1-9)	系统调用/库函数查询	man 2 syscall 查系统调用
elf.h (/usr/include/elf.h)	ELF 结构定义	readelf -h 配合查看
《OSTEP》(Operating Systems: Three Easy Pieces)	三部分：虚拟化、并发、持久化	经典教材，英文免费
相关实验代码	多线程编程参考	生产者-消费者模型

本文档使用指南：

1. 初学时按**阶段0→6**的顺序从上往下看
2. 遇到 bug 先去**第三部分（坑）**查是否有类似情况
3. 复习时从**第五部分（复习策略）**开始，用第四部分（速查表）****辅助
4. 面试前重点过**第二部分的"注意事项"列和第五部分的检查清单**
5. 新增内容直接按阶段归类追加，不破坏已有结构